

Part 7: Smoothing

Summary

Smoothing refers generally to procedures that attempt to extract underlying **signal** from **noise** in a **nonparametric** manner.

The term “nonparametric” means the use of a statistical model that does not make restrictive assumptions. The data are allowed to “speak for themselves” as much as possible.

Here we consider such smoothing procedures as tools for summarizing noisy data.

Basics of Density Estimation

Nonparametric density estimation refers to techniques that estimate the **distribution** for a variable, without assuming any **parametric** form. Examples of parametric forms include the normal distribution, the exponential distribution, etc.

Histograms are a simple example of a nonparametric density estimator. While these are useful, they suffer from limitations, namely the arbitrariness of the binning, and discontinuous (“jagged”) nature of the estimate of a distribution that is typically assumed to be smooth. Smooth estimators also have statistical efficiency advantages over histograms.

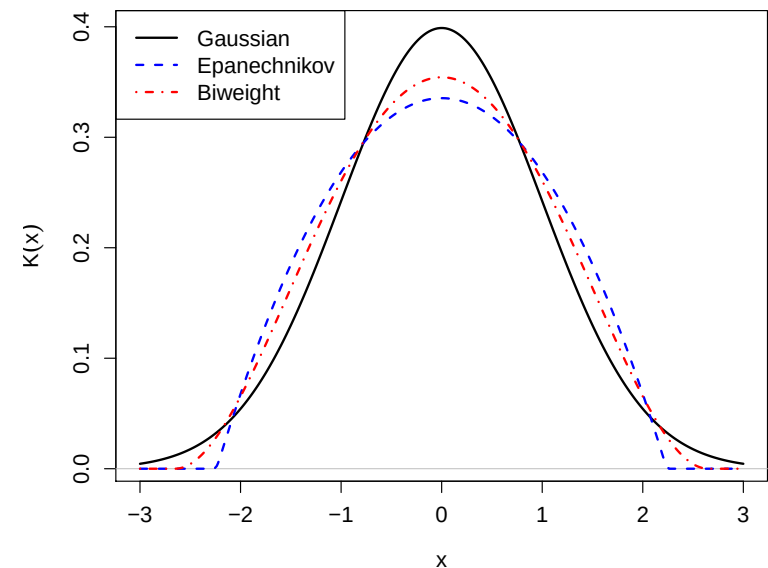
The standard nonparametric approach to density estimation is the **kernel density estimator**. This estimate is constructed by summing a smooth **kernel function** centered at each of the observed data points.

Formally, we write:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

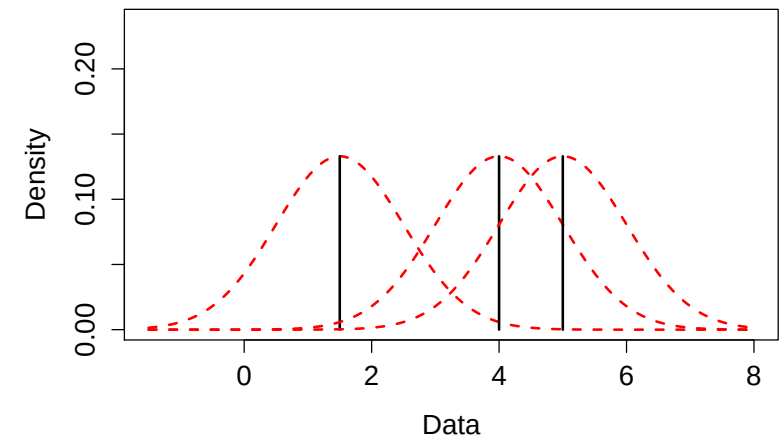
where $K(\cdot)$ is the **kernel function** and h is the **smoothing parameter** or **bandwidth**. The sample is $x_1, x_2, \dots, x_n$.

The kernel function is itself a density, almost always taken to be a smooth density peaked at zero, and symmetric around zero. Three standard examples are shown on the next page.



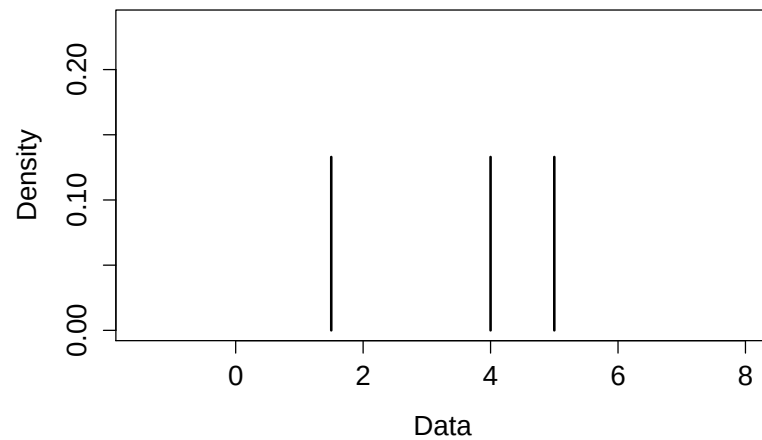
- 1 The choice of the kernel is not too influential on the shape of the density
- 2 estimate. The bandwidth h is influential, however.
- 3 Larger values of h result in a density estimate that is smoother. Smaller
- 4 values of h produce an estimate that is “rough” and “wiggly.”
- 5 As a very simple example to illustrate the concept, imagine my data set
- 6 consisted of three observations: 1.5, 4, and 5. The next three slides illustrate
- 7 the steps going from the raw data to the final estimate. The subsequent
- 8 slides show the effect of varying the bandwidth.

- 1 The gaussian kernel with $h = 1$ shown centered at each observation:



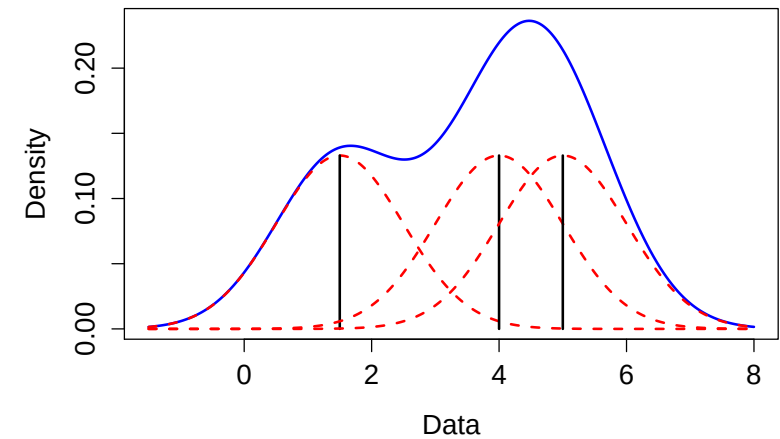
2

- 1 The positions of the three observations:



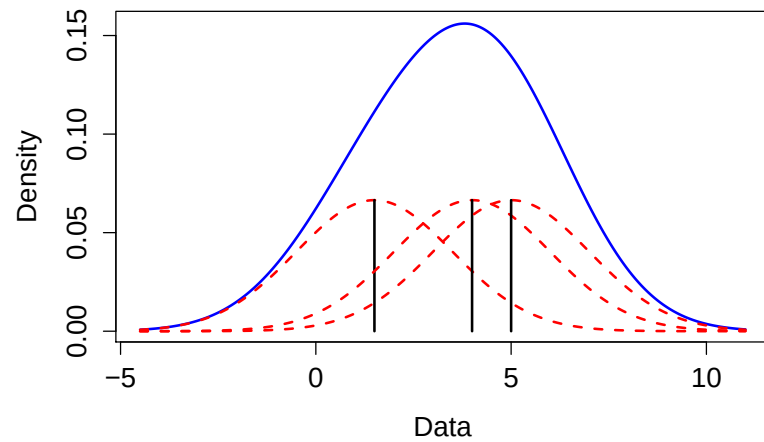
2

- 1 These kernels are summed to create the final estimate:



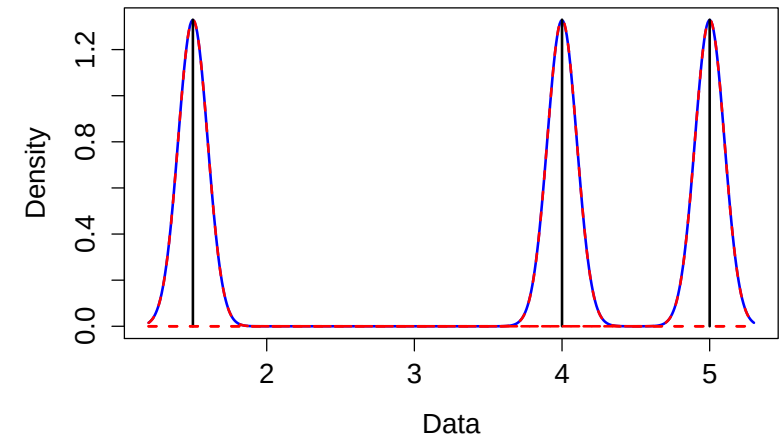
2

- 1 Compare this with the case where $h = 2$:



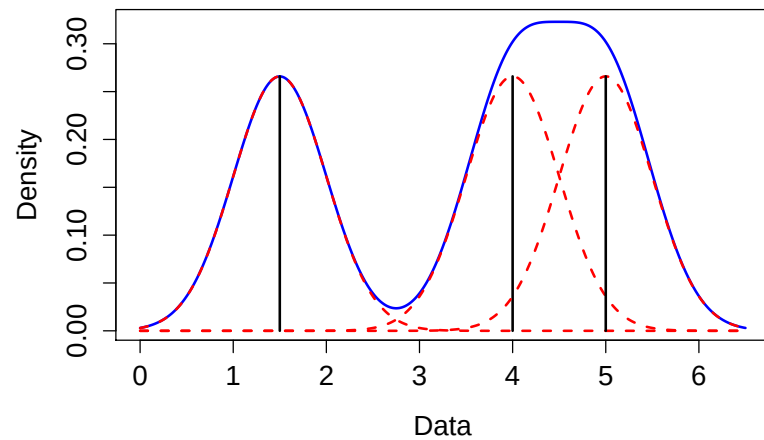
2

- 1 Or when $h = 0.1$:



2

- 1 And when $h = 0.5$:



2

- 1 **Exercise:** What is the relationship between using a Gaussian kernel and
 2 making an assumption that a sample is drawn from a Gaussian (normal)
 3 distribution?

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

1 Rigorous ways of choosing h do exist. See Jones, Marron, and Sheather
2 (1996) for an overview.

3 Briefly, the theory is built on the assumption that the available sample
4 $x_1, x_2, \dots, x_n$ are drawn i.i.d. from some population with density $f(\cdot)$. The
5 criterion that one seeks to minimize is the **mean integrated squared error**:

$$\text{MISE} = E \left[\int (\hat{f}_h(x) - f(x))^2 dx \right].$$

7 As is often the case, consideration is made of the **asymptotic** behavior of
8 the criterion as n increases. One can show that the **asymptotic mean**
9 **integrated squared error** is

$$\text{AMISE} = \frac{R(K)}{nh} + h^4 R(f'') \left(\frac{1}{2} \int x^2 K(x) dx \right)^2$$

11 where

$$R(\phi) = \int \phi^2(x) dx$$

1 The optimal choice for h can then be shown to be

$$h = \left[\frac{R(K)}{n R(f'') \left(\int x^2 K(x) dx \right)^2} \right]^{1/5}$$

3 The **Sheather-Jones (S-J)** approach to finding the bandwidth is to find the
4 h which solves

$$h = \left[\frac{R(K)}{n R(\hat{f}_{g(h)}'') \left(\int x^2 K(x) dx \right)^2} \right]^{1/5}$$

6 where $g(h)$ is some function of h which accounts for the fact that the opti-
7 mal choice of bandwidth for estimating f'' is not the same as the optimal
8 choice for estimating f .

1 The choice of h can also be dictated by less theoretical considerations.

2 In particular, if the motivation is to achieve a summary of a distribution
3 via **smoothing** the distribution to a particular **scale**.

4 One could, for example, smooth a distribution using multiple bandwidth
5 choices, and then using the resulting smooth estimates as input into a
6 model. Formal procedures could then be used to determine how useful
7 each "scale" is to the problem at hand, i.e., a prediction challenge.

8 This idea will be explored through examples later.

1 Kernel Density Estimation in R

2 The basic function for kernel density estimation in R is `density()`.

3 The argument `kernel` specifies the kernel function; the default is
4 "gaussian".

5 The bandwidth is provided using the argument `bw`. The user can specify
6 either a number, or specify a method for determining the bandwidth. For
7 example, the function `bw="SJ"` calculates the S-J bandwidth.

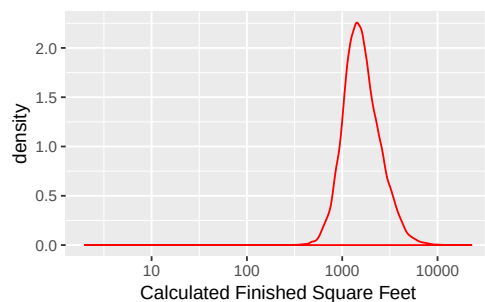
8 The argument `adjust` can be useful when attempting to try multiple
9 scales. The bandwidth utilized is actually equal to `adjust` times the
10 value of `bw`. This makes it easy to fit with, e.g., double the S-J bandwidth,
11 half the S-J bandwidth, etc.

- 1 **Exercise:** Try the following code, and discuss the structure of the output of
 2 `density()`. What happens as the sample size is varied?

```
> x = rnorm(1000)
> densout = density(x, bw="SJ")
> plot(densout)
> curve(dnorm(x), add=T, col="red", lty=2)
```

- 1 There is a function `geom_density()` as part of `ggplot`. It accepts the
 2 same arguments as does `density()`.

```
> ggplot(trainmerged, aes(x=calculatedfinishedsquarefeet)) +
+   geom_density(bw="SJ", color="red") +
+   labs(x="Calculated Finished Square Feet") +
+   scale_x_log10()
```



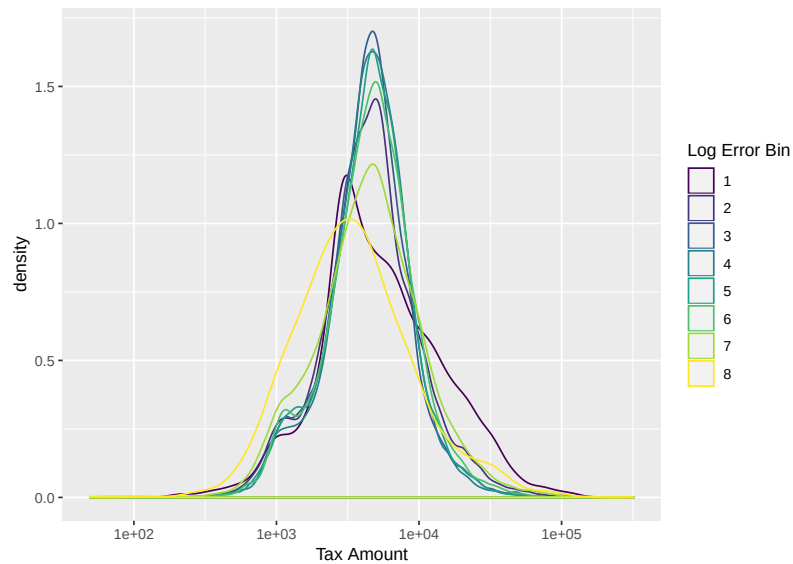
- 1 **Exercise:** How does one interpret the vertical scale “Density” on the pre-
 2 ceding plot?

1 Insights from Density Estimates

- 2 As stated above, a primary motivation for smoothing in general, and non-
 3 parametric density estimation in particular, is to work to separate real fea-
 4 tures (the “signal”) from the uninformative randomness (the “noise”).

- 5 Consider how the next plot presents changes in the distribution of the
 6 amount of property tax over the different log error bins. This builds on
 7 our previous Zillow example.

```
> ggplot(trainmerged, aes(x=taxamount, color=logerrorbin)) +
+   geom_density(bw="SJ") +
+   labs(x="Tax Amount", color="Log Error Bin") +
+   scale_x_log10()
```



1

1 **Exercise:** What conclusion(s) could you draw from the previous plot?

2 _____

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10

1 A kernel density estimate is often utilized to determine an appropriate
2 “named” parametric distribution, if it exists.

3 As an example, use package `quantmod` to obtain the daily data for Kel-
4logg’s (K) from 2010 through 2016:

```
> Kellogg = getSymbols("K",
+   from="2010-1-1",to="2016-12-31", auto.assign=F)
```

5 Then calculate the log daily returns:

```
> ldrK = data.frame(dailyReturn(Ad(Kellogg),
+   type="log"))
```

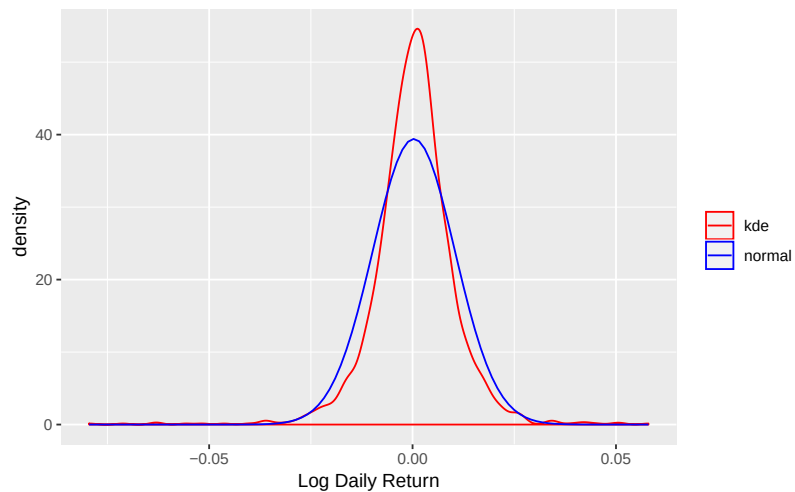
6 The function `Ad()` extracts the “adjusted closing price” column from the
7 data set.

1 Consider the following plot which compares the kernel density estimate
2 with the normal distribution:

```
> ggplot(ldrK,aes(x=daily.returns)) +
+   geom_density(bw="SJ",aes(color="kde")) +
+   stat_function(fun=dnorm,aes(color="normal"),
+     args=list(mean=mean(ldrK$daily.returns),
+       sd=sd(ldrK$daily.returns))) +
+   scale_color_manual(name="",
+     values=c("kde"="red","normal"="blue")) +
+   labs(x="Log Daily Return",title="Data for Kellogg (K)",
+     subtitle="January 2010 through December 2016")
```

Data for Kellogg (K)

January 2010 through December 2016



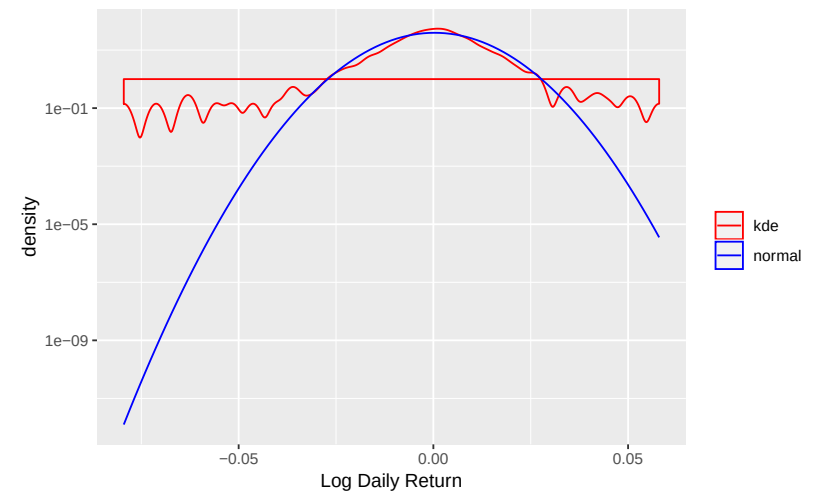
1

1 The plot is not as informative as one on the log scale:

```
> ggplot(ldrK, aes(x=daily.returns)) +
+   geom_density(bw="SJ", aes(color="kde")) +
+   stat_function(fun=dnorm, aes(color="normal"),
+     args=list(mean=mean(ldrK$daily.returns),
+       sd=sd(ldrK$daily.returns))) +
+   scale_color_manual(name="",
+     values=c("kde"="red", "normal"="blue")) +
+   labs(x="Log Daily Return", title="Data for Kellogg (K)",
+     subtitle="January 2010 through December 2016") +
+   scale_y_log10()
```

Data for Kellogg (K)

January 2010 through December 2016



1

1 **Exercise:** Comment on the appropriateness of the normal approximation
2 to log returns in this case.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

1 Converting to the CDF and Percentiles

Smooth density estimates can be transformed into an estimate of the **cumulative distribution function (CDF)** and its inverse (i.e., percentiles).

4 A useful function in R to achieve this is `approxfun()`. It takes `x` and `y`
5 vectors and returns a function which linearly interpolates the given values.
6 Note that the output of `approxfun()` is itself a function.

7 Also, there is a function `integrate()` that will perform numerical inte-
8 gration.

1 Consider the following function `kCDF()`:

```
> kCDF = function(x, res=100, ...)
+ {
+   holddens = density(x,...)
+   interpdens = approxfun(holddens$x, holddens$y,
+                           yleft=0, yright=0)
+   xseq = seq(min(holddens$x),max(holddens$x),length=res)
+   holdout = numeric(res)
+   for(i in 1:res)
+   {
+     holdout[i] = integrate(interpdens, lower=min(holddens$x),
+                             upper=xseq[i], stop.on.error=FALSE)$value
+   }
+   CDF = approxfun(xseq, holdout, yleft=0, yright=1)
+   invCDF = approxfun(holdout, xseq, yleft=NA, yright=NA)
+   list(CDF=CDF, invCDF=invCDF)
+ }
```

1 **Exercise:** Discuss what the function `kCDF()` will do.

2	
3	
4	
5	
6	
7	
8	
9	
10	

1 For this example, let's return to the market structure data that we consid-
2 ered in Parts 4 and 5:

```
> fulldata = read.table("qi_2017_all.csv", sep=",", header=T, quote="")
> fulldata$Date = as.Date(as.character(fulldata$Date), format="%Y%m%d")
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)], fromLast=TRUE),]
> fulldata$McapRank = factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank = factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank = factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank = factor(fulldata$PriceRank, ordered=TRUE)
```

3 We will consider the variable `Cancels`. Split this variable up by the ticker
4 symbol, and exclude those with fewer than 5 observations:

```
> cancelbyticker = split(fulldata$Cancels, fulldata$Ticker)
> cancelbyticker =
+   cancelbyticker[sapply(cancelbyticker, length) >= 5]
```

- 1 Now, we can construct the estimate of the CDF for the symbols using the
2 following:

```
> CDFsbyTicker = lapply(cancelbyticker, kCDF, bw="SJ")
```

- 3 I want to evaluate each of the inverse CDFs at a regular grid of probabilities
4 ranging from 0.1 to 0.9:

```
> pseq = seq(0.1, 0.9, by=0.1)
>
> holdquantiles = matrix(ncol=length(pseq),
+                        nrow=length(CDFsbyTicker))
> for(i in 1:length(CDFsbyTicker))
+ {
+   holdquantiles[i,] = CDFsbyTicker[[i]]$invCDF(pseq)
+ }
```

- 1 **Exercise:** What just happened in this previous code? What was the point
2 of this?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11

1 Smoothing Relationships

- 2 Smoothing techniques are also very commonly applied to situations where
3 one variable is naturally thought of as a **response**, i.e., a quantity to be
4 predicted, and other variables are considered as **predictors**.

- 5 The discussion that follows presents an overview of **nonparametric** regres-
6 sion. The focus is placed on constructing useful **summaries** that will lead
7 to better understanding of the relationship between the response and the
8 predictor(s).

1 Example: Pricing Options

- 2 A **European call option** on a stock is a contract that gives the holder the
3 right to purchase that stock at the named **strike price** on the **expiration**
4 **date**.

- 5 A **American call option** is the same, but allows the holder to purchase the
6 stock at that price **on or before** the expiration date.

- 7 The classic **Black-Scholes Theory** for option pricing establishes a price for
8 a European option as a function of the following: the strike price, the time
9 to expiration, the current asset price, the volatility of the price, and the
10 current **risk-free rate of return**.

- 1 We will consider a data set consisting of 1,000 call options sampled on
- 2 September 29, 2017.
- 3 To generate this sample, NYSE stock symbols were randomly chosen, and
- 4 then one currently-listed call option was randomly chosen for each equity,
- 5 weighted by the volume of trading.
- 6 Information recorded for each were as follows: The strike price, the time
- 7 to expiration, the current asset price, the historical volatility (based on
- 8 daily returns of 30 most recent trading days), the **implied volatility** and the
- 9 Black-Scholes price for the option (with a risk-free rate of zero assumed).

- 1 The full R code to generate such a sample can be found on Canvas. There
- 2 are some important components worth pointing out:
- 3 The package `quantmod` includes a function `getOptionChain()` which
- 4 allows one to easily obtain the current options information for a ticker
- 5 symbol.
- 6 The package `fOptions` includes `GBSOption()` and `GBSVolatility()`,
- 7 which can be used to calculate the Black-Scholes price and the implied
- 8 volatility for an option, respectively.
- 9 The package `TTR` includes a function `stockSymbols()` which will re-
- 10 turn all of the ticker symbols (along with other useful information) for the
- 11 AMEX, NASDAQ, or NYSE exchange.

- 1 **Exercise:** The following appears as part of the provided code.

```
> holdout = try(getOptionChain(cand, NULL, auto.assign=F), TRUE)
>
> if("try-error" %in% class(holdout) || length(holdout) == 0)
+ {
+   next
+ }
```

- 2 What is the purpose of this syntax?

- 1 This sample can be found in the Data Sets area on Canvas, with the name
- 2 `optionssample09302017.txt`.

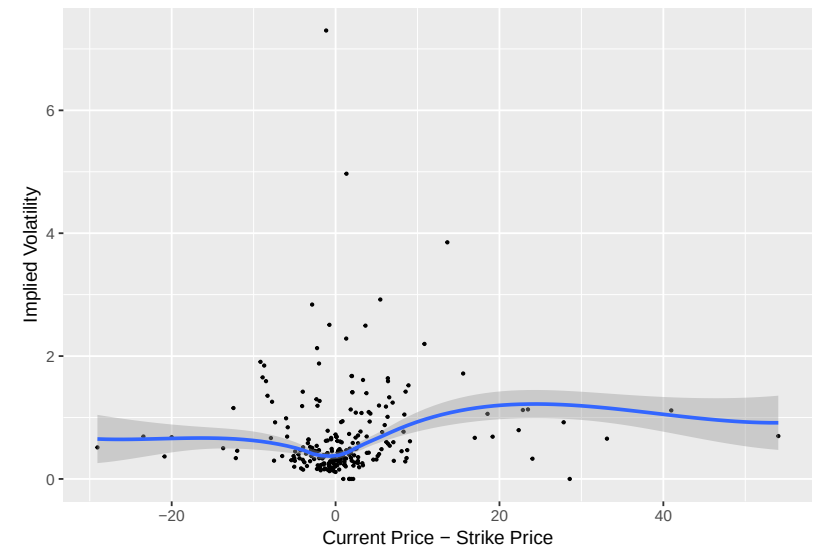
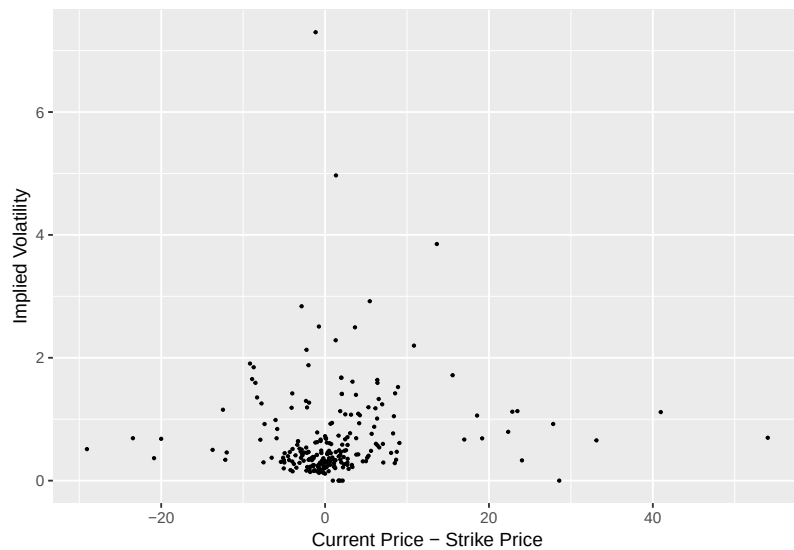
```
> optionsdata = read.table("optionssample09302017.txt",
+                           sep="," , header=T)
```

- 3 Our objective is to explore the relationships between the current ask price
- 4 for the option and the available variables.
- 5 This is an “empirical” alternative to Black-Scholes theory, whose limita-
- 6 tions are well-known, mainly due to the failure of the underlying normal-
- 7 ity assumption.

1 A classic manifestation of the failure of Black-Scholes is the so-called
 2 **volatility smile**. Under the theory, a plot of implied volatility versus how
 3 far “in the money” the option currently is should be flat for options with
 4 the same expiration date.

5 Let’s create this plot in our case. Is there evidence of the smile here?

```
> optionsdata$inmoney = optionsdata$curprice -
+   optionsdata$strike
> optionsdata20 = filter(optionsdata, timetoexpiry==20)
> ggplot(optionsdata20, aes(x=inmoney, y=implvol)) +
+   geom_point(size=0.5) +
+   labs(x="Current Price - Strike Price",
+        y="Implied Volatility")
```



1 Local Linear Regression

2 The figure on the previous page shows a **nonparametric regression** or **non-**
 3 **parametric smooth** of the scatter plot.

4 There are many variants of nonparametric regression, but here we focus
 5 on **local linear regression**, a version of **local polynomial regression**. This
 6 approach has proven to be very powerful, and possesses many strong the-
 7 oretical properties to justify its use.

8 Briefly stated, this procedure works by fitting a sequence of linear models:
 9 Each is fit not to the entire data set, but to only data within a **neighborhood**
 10 of a target point. The size of this neighborhood is a **smoothing param-**
 11 **eter**: Large neighborhoods yield a large degree of smoothing, while small
 12 neighborhoods result in minimal smoothing.

1 Our model here is that we observe (x_i, Y_i) for $i = 1, 2, \dots, n$ and that

$$Y_i = f(x_i) + \epsilon_i$$

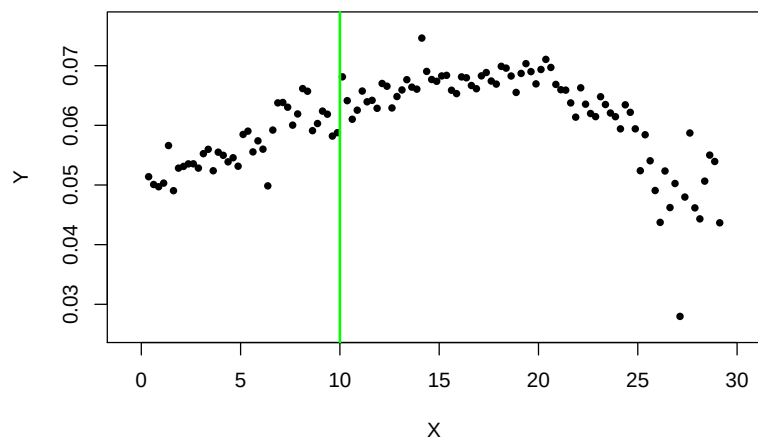
2
3 where the ϵ_i are iid with mean zero and variance σ^2 . Assuming that the ϵ_i
4 are normal will lead to further nice properties, but this development does
5 not require that assumption.

6 In order to construct the local linear regression estimate of $f(\cdot)$, it is best
7 to consider a sequence of steps **for each fixed x_0 at which $f(\cdot)$ will be esti-**
8 **mated.**

9 (For context on these data, see Ruppert and Matteson, Section 11.3.)

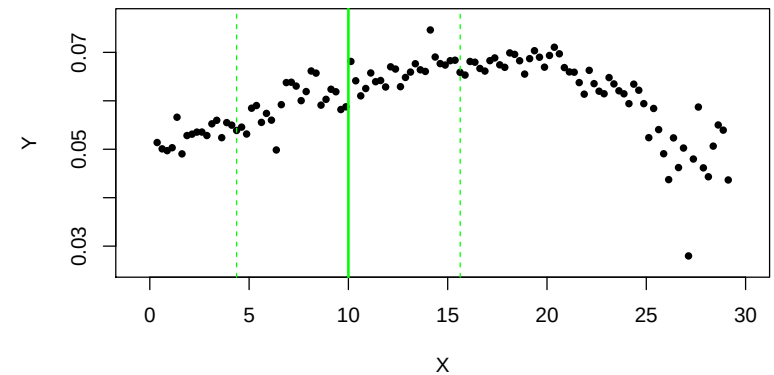
1 **Step One: Fix the target point x_0 .**

2 Our objective is to estimate the regression function at x_0 .



1 **Step Two: Create the neighborhood around x_0 .**

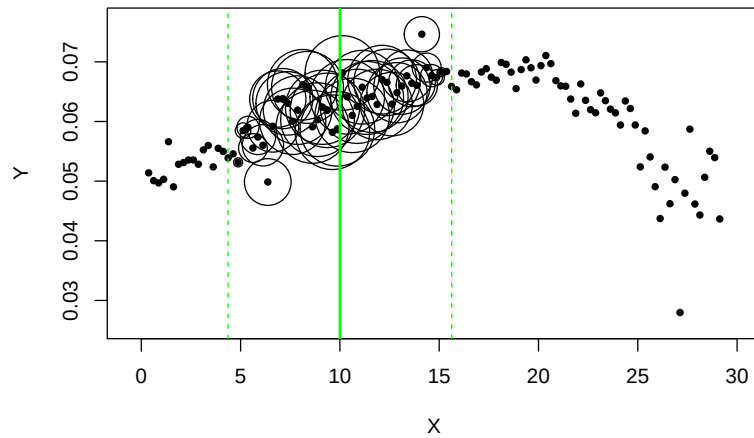
2 A common way to choose the neighborhood size is to choose is large
3 enough to capture proportion α of the data. This parameter α is often
4 called the **span**. A typical choice is $\alpha \approx 0.5$.



1 **Step Three: Weight the data in the neighborhood.**

2 Values of x which are close x_0 will receive a larger weight than those far
3 from x_0 . Denote by w_i the weight placed on observation i . The default
4 choice is the **tri-cube weight function**:

$$w_i = \begin{cases} \left(1 - \left|\frac{x_i - x_0}{\max \text{ dist}}\right|^3\right)^3, & \text{if } x_i \text{ in the neighborhood of } x_0 \\ 0, & \text{if } x_i \text{ is not in neighborhood of } x_0 \end{cases}$$

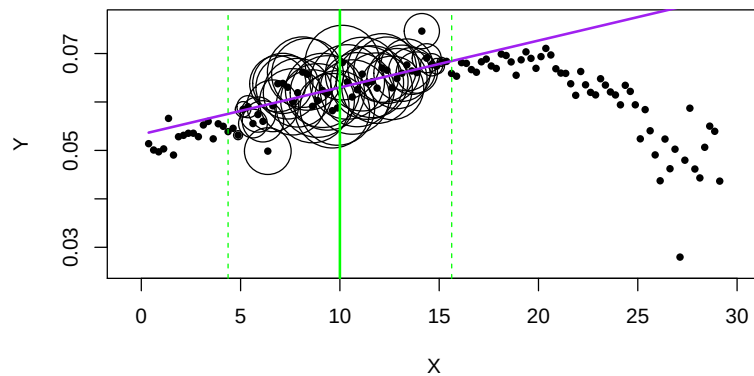


1

1 Step Four: Fit the local regression line.

2 This is done by finding β_0 and β_1 to minimize the weighted sum of squares

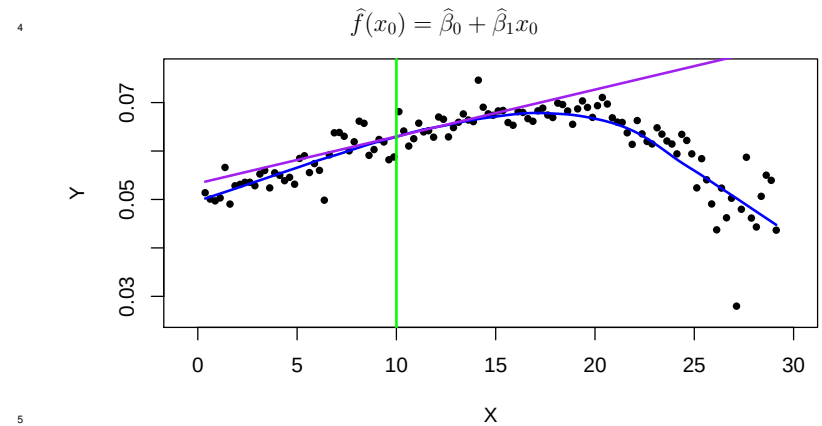
$$\sum_{i=1}^n w_i (y_i - (\beta_0 + \beta_1 x_i))^2$$



4

1 Step Five: Estimate $f(x_0)$.

2 This is done using the fitted regression line to estimate the regression function at x_0 :



5

1 This is implemented in R using `loess()`:

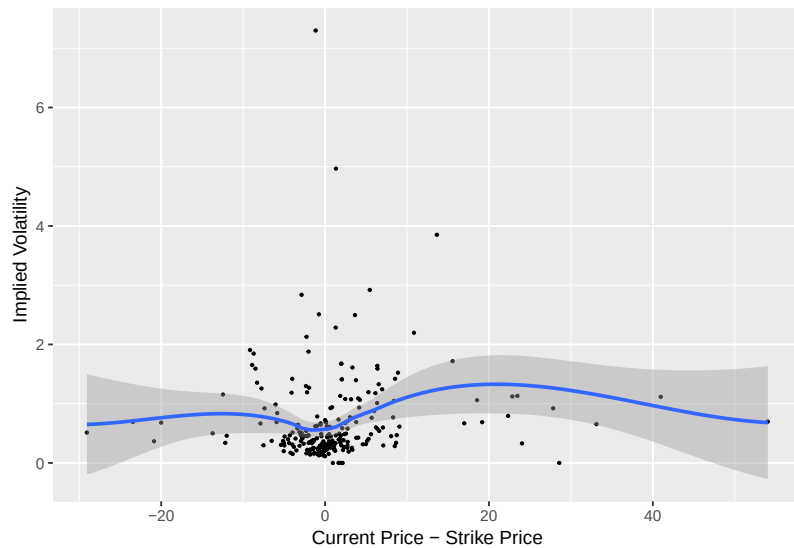
```
> holdlo = loess(implvol ~ inmoney,
+               data=optionsdata20, degree=1)
```

2 Setting `degree = 1` is necessary in order to get local linear models. You can use the default `degree = 2` to get local quadratic fits, but this is usually unnecessary.

5 `ggplot` makes it easy to place the local linear regression fit onto the plot.

6 Note how `degree=1` is passed on to `loess()`:

```
> ggplot(optionsdata20, aes(x=inmoney, y=implvol)) +
+   geom_point(size=0.5) + geom_smooth(method="loess",
+   method.args=list(degree=1)) +
+   labs(x="Current Price - Strike Price",
+   y="Implied Volatility")
```



1 Smoothing parameter selection in Regression

2 Just like the choice of the bandwidth in kernel density estimation, the
 3 smoothing parameter in nonparametric regression controls how “smooth”
 4 the resulting estimate is: A smaller value leads to a “rougher” estimate, a
 5 larger value gives a “smoother” estimate.

6 The argument `span` to `loess()` controls the span, as defined above. The
 7 default value is 0.75.

8 **Exercise:** Try refitting the model above with smaller and larger values of
 9 `span`.

1 Automated approaches to smoothing parameter selection do exist. Dif-
 2 ferent ideas are available, but a very popular approach is based on **cross**
 3 **validation**.

4 The motivation behind cross validation is that we seek a regression func-
 5 tion that makes accurate predictions for **new** observations, not for those
 6 that are used to fit the model. A regression function that makes accurate
 7 predictions only for training data is said to be **overfit**. Overfitting is a seri-
 8 ous problem, and is a leading drawback to the use of flexible nonparamet-
 9 ric and machine learning methods.

1 **Exercise:** Discuss how nonparametric regression, and the choice of the
 2 smoothing parameter, leads to increased risk of overfitting. Does choos-
 3 ing the smoothing parameter to minimize residual sum of squares seem
 4 like a good idea?

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

1 The **residual sum of squares** in this case is

$$2 \quad \text{RSS} = \sum_{i=1}^n (Y_i - \hat{f}_h(x_i))^2$$

3 where $\hat{f}_h(x_i)$ is the estimate of the regression function $f(\cdot)$ when smoothing
4 parameter h is used.

5 Instead of choosing h to minimize RSS, we minimize the **leave-one-out**
6 **cross validation score**:

$$7 \quad \text{LOOCV} = \sum_{i=1}^n (Y_i - \hat{f}_h^{(-i)}(x_i))^2$$

8 where $\hat{f}_h^{(-i)}(x_i)$ is the estimate of the regression function **when the i^{th} ob-**
9 **servation is excluded from the training set.**

10 The idea here is that, by leaving out observation i , we are treating it as if
11 it were a “new” observation, and testing to see how well this choice of h
12 does in predicting the response.

1 There is another R function `loess.as()`, which is part of the package
2 `fANCOVA`, which has built in selection of the span.

3 The syntax is

```
> holdlo2 = loess.as(optionsdata20$inmoney,
+ optionsdata20$simplvol, criterion = "gcv")
```

4 With this function the default is `degree = 1`. Setting `criterion="gcv"`
5 uses **generalized cross validation**, which is an approximation to leave-one-
6 out cross validation.

7 The optimal span can be found via

```
> holdlo2$pars$span
[1] 0.7038925
```

1 Robust Fits

2 **Exercise:** Reconsider the smooth shown in the Figure on Slide 53 above.

3 Does it appear to be a good fit?

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

1 The problem in the fit is that the extreme values are overly **influential**.
2 Least squares is particularly sensitive to these extremes; the large response
3 values are “pulling up” the estimate.

4 Fortunately, `loess()` and `loess.as()` allow one to move away from
5 least squares, by setting the argument `family="symmetric"` instead of
6 the default `family="gaussian"`.

```
> holdlo3 = loess.as(optionsdata20$inmoney,
+ optionsdata20$simplvol, family="symmetric",
+ criterion = "gcv")
```

- 1 **Exercise:** What is the reasoning behind using the name `family`, with
 2 value `symmetric`, to specify a robust fit?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

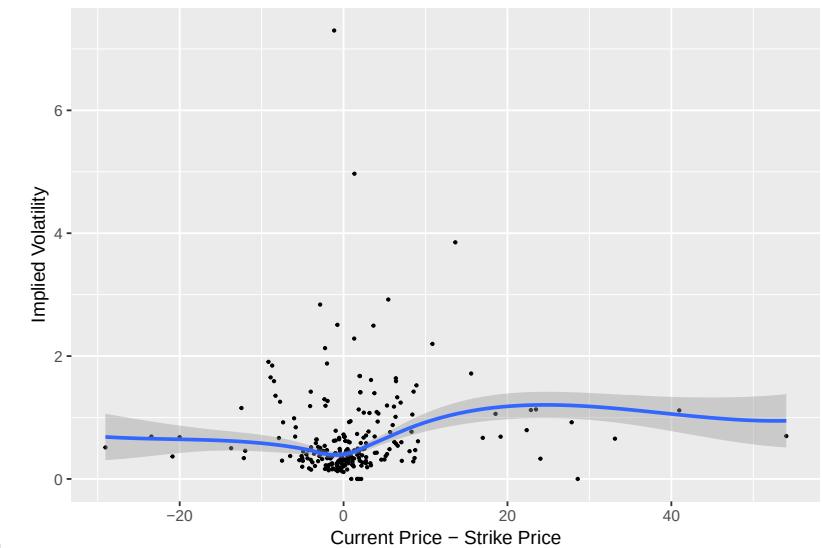
9 _____

10 _____

11 _____

- 1 Now, we refit with this span, using `family="symmetric"`:

```
> ggplot(optionsdata20, aes(x=inmoney, y=implvol)) +
+   geom_point(size=0.5) +
+   geom_smooth(method="loess", span=holdlo3$pars$span,
+     method.args=list(degree=1,
+       family="symmetric")) +
+   labs(x="Current Price - Strike Price",
+     y="Implied Volatility")
```



1

1 Confidence Interval

- 2 The depicted gray region around the regression function estimate is a **95%**
 3 **pointwise confidence band** for the regression function.
- 4 The **proper** interpretation is as follows: For each predictor value x , the
 5 gray region displays a 95% confidence interval for $f(x)$.
- 6 Here are two crucial things to understand regarding the band:
- 7 1. The shaded area does **not** show a **simultaneous** 95% confidence band
 8 for the **entire** regression function.
 - 9 2. The shaded area does **not** show 95% **prediction intervals** for new ob-
 10 servations.

- 1 **Exercise:** Discuss the two misconceptions given above, and how they differ from the real interpretation.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

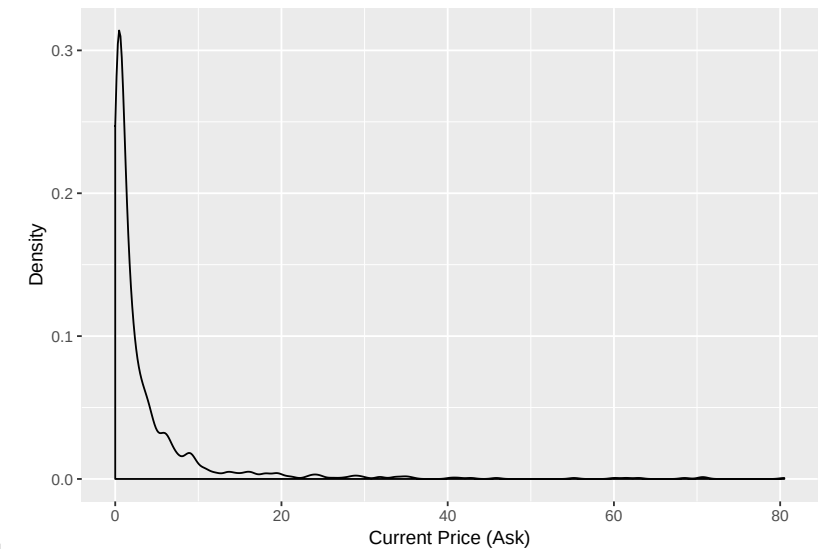
11 _____

1 Exploring Relationships

- 2 Now we will dig a little deeper into the options sample, and explore relationships between the relevant properties of an option and its current ask price.

- 5 First, consider the distribution of the prices. There is a notable skew. In what follows we will work with the logarithm of the ask price.

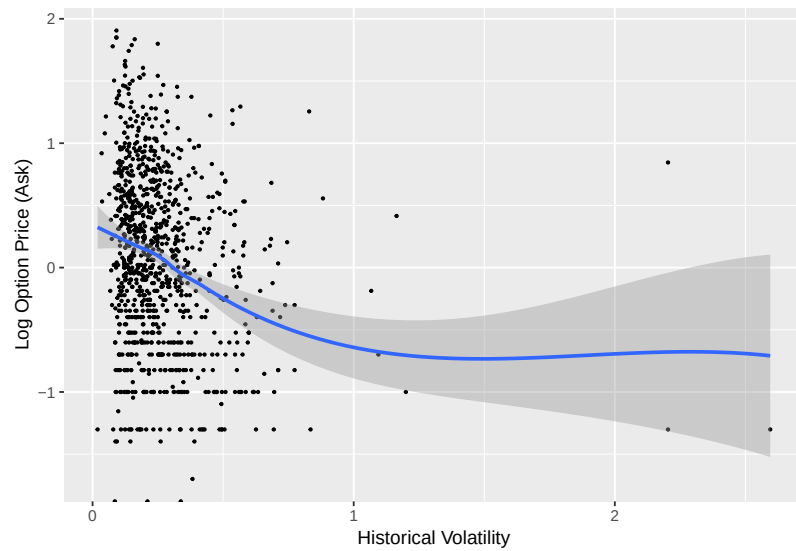
```
> ggplot(optionsdata, aes(x=ask)) + geom_density() +
+ labs(x="Current Price (Ask)", y="Density")
```



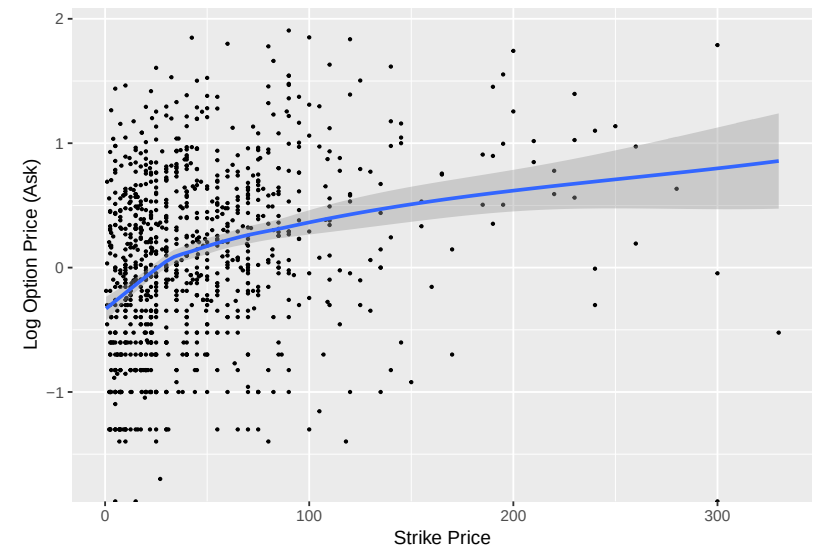
1

- 1 The next plots compare historical volatility, time to expiration, and strike price to the price. The code is only shown here for the first plot:

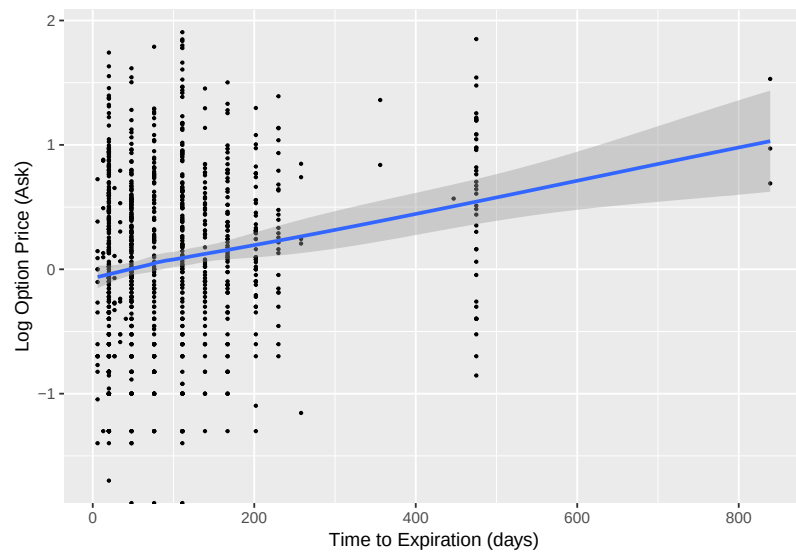
```
> ggplot(optionsdata, aes(x=histvol, y=log10(ask))) +
+ geom_point(size=0.5) +
+ geom_smooth(method="loess",
+             method.args=list(degree=1)) +
+ labs(x="Historical Volatility",
+      y="Log Option Price (Ask)")
```



1



1



1

- 1 **Exercise:** One might guess that there is a strong relationship between the
 2 price of the option and how far “in the money” it currently is, i.e., the
 3 difference between the current price of the stock and its strike price. Con-
 4 struct a plot to explore this.

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

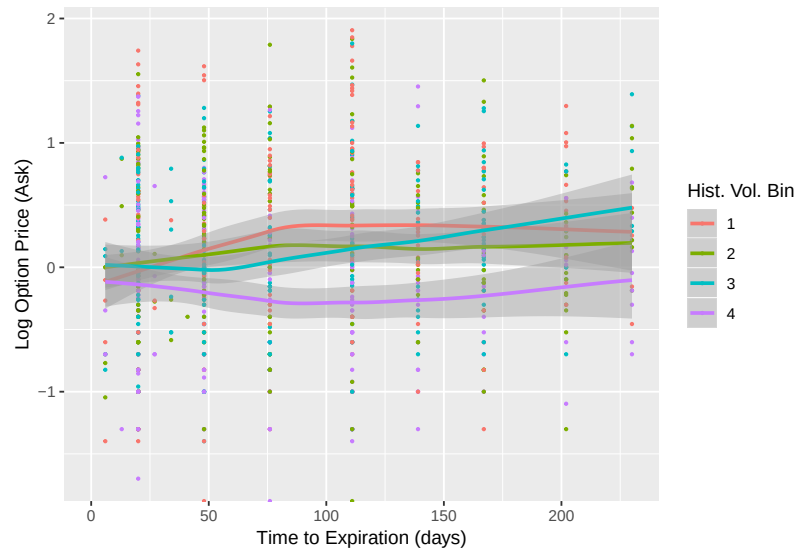
11 _____

- 1 Next we will construct a plot attempting to explore how a pair of variables
- 2 relate to the price.

- 3 Let's bin the historical volatility into four groups:

```
> breaks = c(-Inf, quantile(optionsdata$histvol,
+                           c(0.25,0.5,0.75)), Inf)
> optionsdata$histvolbin = cut(optionsdata$histvol,
+                              breaks, labels=c(1,2,3,4))

> ggplot(optionsdata, aes(x=timetoexpiry,y=log10(ask),
+                          color=histvolbin)) +
+   geom_point(size=0.5) +
+   geom_smooth(method="loess",
+               method.args=list(degree=1)) +
+   labs(x="Time to Expiration (days)",
+        y="Log Option Price (Ask)",
+        color="Hist. Vol. Bin") +
+   xlim(0,230)
```



- 1 **Exercise:** Interpret the graph on the previous page.

- 2 _____
- 3 _____
- 4 _____
- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____
- 10 _____